



2025-2026

MODULO DESPLIEGUE DE APLICACIONES WEB ACTIVIDAD

LUZ A PD
IES MARÍA ENRIQUEZ
DAW 2



Unidad 2 - Práctica Evaluación Continua 2

Instrucciones:

- Una vez finalizada, entrega la actividad en formato PDF en la tarea de Aules. Recuerda que los enunciados también deben aparecer en el PDF.
- Al final del ejercicio se encuentra la rúbrica de evaluación

RA's:

- RA2 - *Implanta aplicaciones web en servidores web, evaluando y aplicando criterios de configuración para su funcionamiento seguro.*
- RA3 - *Implanta aplicaciones web en servidores de aplicaciones, evaluando y aplicando criterios de configuración para su funcionamiento seguro.*

1. Dado el siguiente fichero de docker-compose, explica con tus palabras qué contiene de forma general. Después punto por punto explica qué se está configurando. Puedes hacerlo sobre el propio código en formato de comentarios.

Este es un fichero Docker compose que se usa para dar instrucciones a Docker para la creación de contenedores basados en las instrucciones detalladas, es un tipo de fichero que se crea para la configuración que hará Docker, en general es un conjunto de servicios (imágenes) que se levantarán en contenedores separados y formarán un entorno de desarrollo con los servicios necesarios para este.

```
services://indica cuales serán los contenedores que tendrá
app://nuestra app
image: php:8.3-apache //imagen de apache sobre la cual se crea el contenedor
container_name: php_app //nombre que le asignamos al contenedor
volumes:// crea un volumen para la aplicación en la path indicada
- ~/Escritorio/projectesPHP:/var/www/html
ports: configura el puerto que utilizará este contenedor
- "8080:80"
depends_on://indica que se debe levantar antes que db
- db
restart: always// se usa para que el contenedor se reinicie si algo falla

db://aquí inicia otro contenedor uno de db
image: mysql:8.0//imagen de mysql
container_name: mysql_db//nombre del contenedor
environment: //para declarar las variable de entorno credenciales con las cuales accederemos
MYSQL_ROOT_PASSWORD: rootpass//variable de entorno password
MYSQL_DATABASE: testdb//var entorno
MYSQL_USER: user//var entorno
MYSQL_PASSWORD: userpass
volumes://define la ruta de la carpeta en la cual se almacenará
- db_data:/var/lib/mysql
restart: always//asegura estabilidad del servicio
```

`phpmyadmin://`nombre del servicio o contenedor
`image:` `phpmyadmin/phpMyAdmin` //imagen de la cual se parte
`container_name:` `phpMyAdmin`//nombre del contenedor
`environment:` variables para esta aplicación o variables de entorno
`PMA_HOST:` `db` // nombre del host
`PMA_USER:` `root`//usuarios del host
`PMA_PASSWORD:` `rootpass`//pw del usuario
`ports://`definición de puertos por los cuales accederemos
`- "8081:80"`
`depends_on:` debe instalarse antes que la db
`- db`
`restart:` `always`//siempre debe estar disponible, se reinicia si algo falla
`volumes:`
`db_data://`nombre del volumen general de la aplicacion

2. Monta un entorno web completo utilizando Docker Compose, simulando un pequeño entorno de desarrollo profesional.

El objetivo es desplegar tres servicios que trabajen juntos:

- a. Un **servidor web Apache** con PHP, que servirá una aplicación escrita en PHP desde la carpeta local `./web`.
 - i. El servidor debe ser accesible desde el navegador a través del puerto 8080.
 - ii. Los archivos de la aplicación deben residir en tu máquina, de forma que cualquier cambio en los archivos PHP se vea reflejado al instante en el contenedor.
- b. Una **base de datos MySQL**, que almacenará los datos de la aplicación
 - i. La base de datos se llamará `blog`.
 - ii. Debe tener una contraseña de `root` (`secret`) y conservar los datos entre ejecuciones usando un **volumen persistente**.
- c. **phpMyAdmin**, para administrar la base de datos
 - i. Debe ser accesible desde el navegador en el puerto **8081**.

La aplicación PHP deberá conectarse a la base de datos y mostrar un mensaje indicando si la conexión ha sido exitosa o no.

Cuando todo funcione, podrás acceder a:

Tu aplicación en `http://localhost:8080`

phpMyAdmin en `http://localhost:8081`

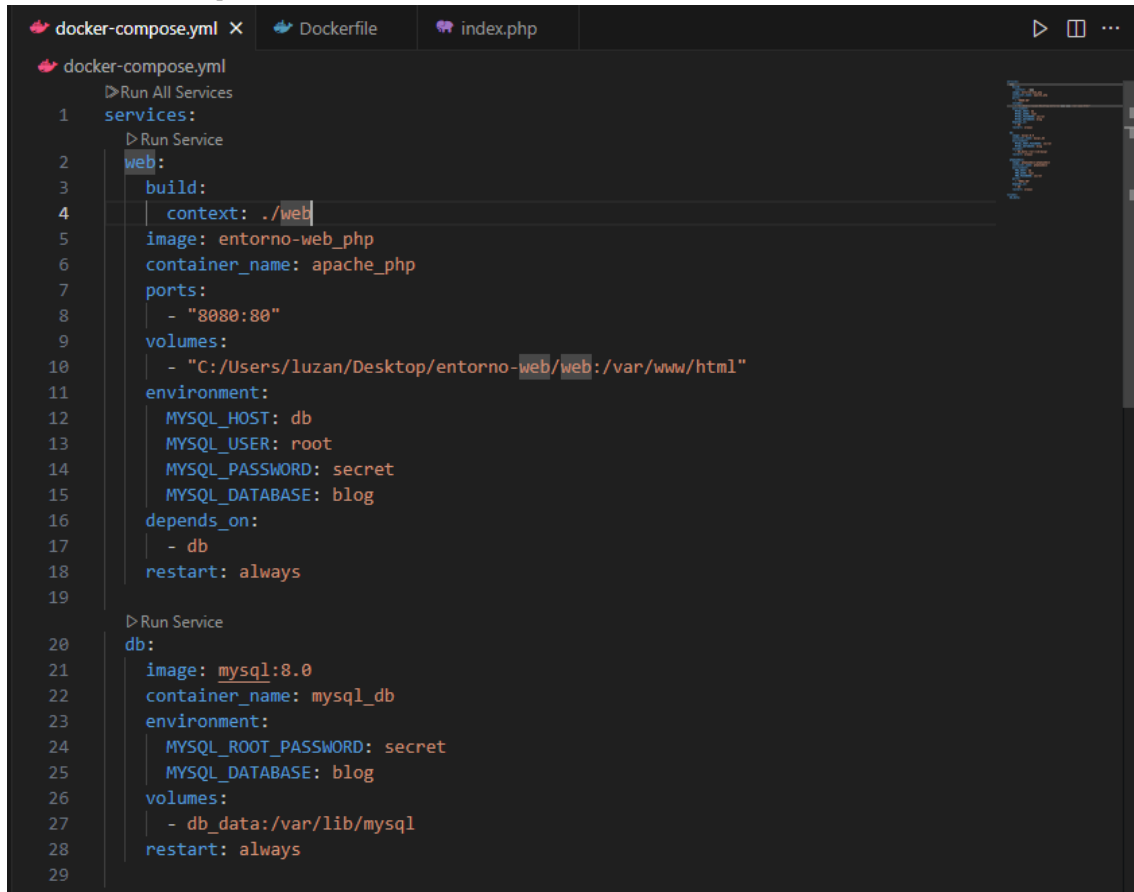
El fichero PHP para probar la conexión puede ser el siguiente:

```
<?php
```

```
//Los parámetros que aparecen dentro de getenv son las variables de entorno
$servername = getenv('MYSQL_HOST');
$username = getenv('MYSQL_USER');
$password = getenv('MYSQL_PASSWORD');
$dbname = getenv('MYSQL_DATABASE');
$conn = new mysqli($servername, $username, $password, $dbname);
if ($conn->connect_error) {
    die("<h2 style='color:red;'>Conexión fallida: " . $conn->connect_error .
"</h2>");
}
echo "<h2 style='color:green;'>Conexión exitosa a la base de datos '$dbname'
en el host '$servername'</h2>";
$conn->close();
?>
```

Una vez te funcione todo, añade capturas de:

- Fichero compose.yml



```
docker-compose.yml
1  services:
2    web:
3      build:
4        context: ./web
5      image: entorno-web_php
6      container_name: apache_php
7      ports:
8        - "8080:80"
9      volumes:
10       - "C:/Users/luzan/Desktop/entorno-web/web:/var/www/html"
11      environment:
12        MYSQL_HOST: db
13        MYSQL_USER: root
14        MYSQL_PASSWORD: secret
15        MYSQL_DATABASE: blog
16      depends_on:
17        - db
18      restart: always
19
20   db:
21     image: mysql:8.0
22     container_name: mysql_db
23     environment:
24       MYSQL_ROOT_PASSWORD: secret
25       MYSQL_DATABASE: blog
26     volumes:
27       - db_data:/var/lib/mysql
28     restart: always
29
```

```

30  Run Service
31  phpmyadmin:
32    image: phpmyadmin/phpmyadmin
33    container_name: phpmyadmin
34    environment:
35      PMA_HOST: db
36      PMA_USER: root
37      PMA_PASSWORD: secret
38    ports:
39      - "8081:80"
40    depends_on:
41      - db
42    restart: always
43  volumes:
44    db_data:
45

```

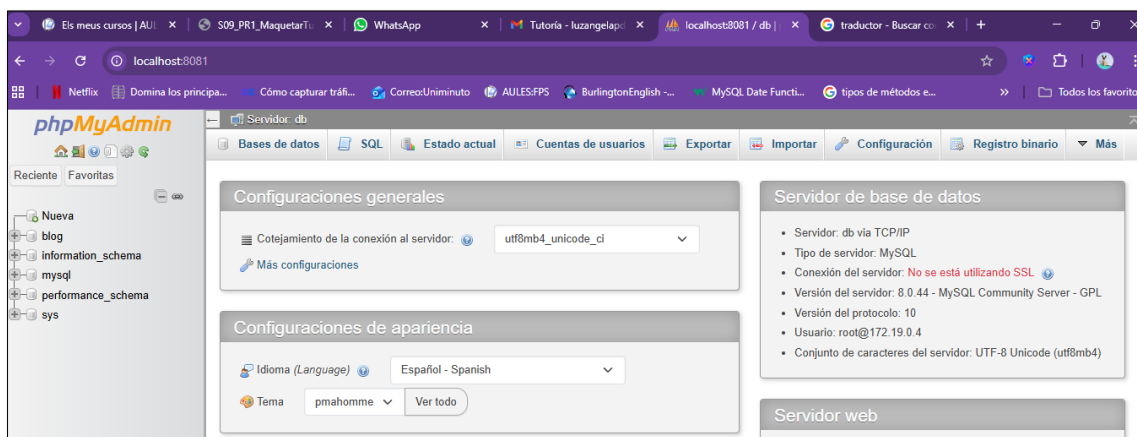
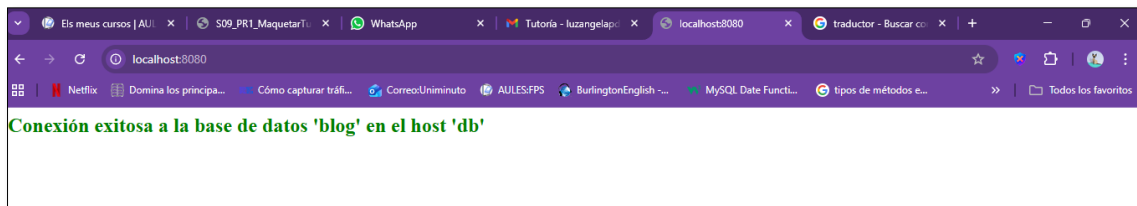
– **docker ps** donde se vean los tres contenedores funcionando simultáneamente.

```

C:\Users\luzan\Desktop\entorno-web>docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED
STATUS        PORTS                               NAMES
18a123cb483c   entorno-web_php                    "docker-php-entrypoi..." 5 minutes a
go Up 4 minutes 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp  apache_php
03b08792680c   phpmyadmin/phpmyadmin              "/docker-entrypoint. ..." 5 minutes a
go Up 4 minutes 0.0.0.0:8081->80/tcp, [::]:8081->80/tcp  phpmyadmin
2bf8c5792491   mysql:8.0                          "docker-entrypoint.s ..." 5 minutes a
go Up 5 minutes 3306/tcp, 33060/tcp          mysql_db
C:\Users\luzan\Desktop\entorno-web>

```

– Capturas de acceso a través del navegador a web y phpmyadmin (dentro).



MODULO DESPLIEGUE DE APLICACIONES WEB

UND 2

Practica Docker Avanzado

Page | 6

3. Utilizando Kubernetes, crea la misma infraestructura que en el ejercicio anterior y lánzalo teniendo 3 pods para la parte del servidor web. Añade una captura de los pods funcionando y del manifesto yaml.

```
C:\Users\luzan\Desktop\entorno-web\web>kubectkl get pods
NAME                                READY   STATUS    RESTARTS   AG
db-deployment-798d889545-pm4v5      1/1     Running   0           20
s
phpmyadmin-deployment-55b88f4867-gpv6p 1/1     Running   0           5m
2s
web-deployment-7584b5bd74-57l5j      1/1     Running   0           3m
28s
web-deployment-7584b5bd74-fzkc9      1/1     Running   0           5m
3s
web-deployment-7584b5bd74-wx2tq      1/1     Running   0           3m
40s
C:\Users\luzan\Desktop\entorno-web\web>
```

```
# DEPLOYMENT WEB (PHP-APACHE)

apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-deployment
spec:
  replicas: 3 # tres pods para la parte del servidor web
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: web
          image: php:8.2-apache
          ports:
            - containerPort: 80
          env:
```

```
- name: MYSQL_HOST
  value: mysql-service
- name: MYSQL_USER
  value: root
- name: MYSQL_PASSWORD
  value: secret
- name: MYSQL_DATABASE
  value: blog

---

# SERVICE WEB

apiVersion: v1
kind: Service
metadata:
  name: web-service
spec:
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 80
  type: LoadBalancer # permite acceder desde localhost:80 (balancea
entre los 3 pods)

---

# DEPLOYMENT MYSQL

apiVersion: apps/v1
kind: Deployment
metadata:
  name: db-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
        - name: mysql
          image: mysql:8.0
```

```
    env:
      - name: MYSQL_ROOT_PASSWORD
        value: secret
      - name: MYSQL_DATABASE
        value: blog
    ports:
      - containerPort: 3306

---

# SERVICE MYSQL

apiVersion: v1
kind: Service
metadata:
  name: mysql-service
spec:
  selector:
    app: mysql
  ports:
    - port: 3306
      targetPort: 3306
  clusterIP: None # permite conexión directa interna desde otros pods
                  (como el web)

---

# DEPLOYMENT PHPMYADMIN

apiVersion: apps/v1
kind: Deployment
metadata:
  name: phpmyadmin-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: phpmyadmin
  template:
    metadata:
      labels:
        app: phpmyadmin
    spec:
      containers:
        - name: phpmyadmin
          image: phpmyadmin/phpmyadmin
```

```
    env:
      - name: PMA_HOST
        value: mysql-service
      - name: PMA_USER
        value: root
      - name: PMA_PASSWORD
        value: secret
    ports:
      - containerPort: 80

---
# SERVICE PHPMYADMIN

apiVersion: v1
kind: Service
metadata:
  name: phpmyadmin-service
spec:
  selector:
    app: phpmyadmin
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30001 # acceder desde http://localhost:30001
  type: NodePort
```